PRESCRIPTIVE GUIDANCE

Agentic design patterns

A pattern language for where models decide and code enforces.

Twenty patterns, eight larger, twelve smaller, for the problems you hit once your agent stops being a demo. Each one names a context, a problem, the forces in play, a solution, and what the system looks like after you apply it.

FORMAT	LARGER PATTERNS	SMALLER PATTERNS	EXAMPLE RUNTIME
Pattern language	8 chapters (3–10)	12 appendix patterns	Anthropic via Amazon Bedrock

UPDATED
1 May 2026

★★ Pattern shape is settled ★ Pattern recurs reliably, shape being refined

Contents

FRONT MATTER

— Author

— Abstract

— How to read this

CHAPTERS

1 Introduction

2 The pattern template

3 Agentic Loop

4 Deterministic Override

5 Tool Gate

6 Validation-Retry Loop

7 Structured Error Classification

8 Confidence-Based Routing

9 Review Pass

10 Coordinator-Subagent Hierarchy

11 Closing

SMALLER PATTERNS

A.1 Tool Description Disambiguation

A.2 Session Facts

A.3 Multi-Concern Decomposition

A.4 Nullable Fields

A.5 Other + Detail Enum

A.6 Few-Shot Format Examples

A.7 Domain Partitioning

A.8 Parallel Execution

A.9 Error Propagation

A.10 Conflict Detection

A.11 Lost-in-the-Middle Mitigation

A.12 Strict Tool Use

BACK MATTER

— References

BP

AUTHOR

Bryn Price

Principal Solutions Architect, Amazon Web Services

These patterns come from building agentic systems with AWS customers and watching the same bugs show up in different codebases: a model picks the wrong tool, a refund runs before the eligibility check, a subagent times out and the whole run dies. The language here is my attempt to name what keeps recurring, cross-checked against prior art from adjacent fields (policy engines, workflow DAGs, RPC error taxonomies, structured validation) so the names compose with ideas people already know.

This guidance pairs with the workshop *Agentic Design Patterns with Claude*. Every pattern in here has runnable code in the workshop; every exercise in the workshop maps back to a pattern.

Abstract

This is a pattern language for agent engineering. It names twenty recurring shapes, eight larger, twelve smaller, that show up again and again when you build systems in which a language model decides what to do, calls tools, and continues. The patterns have been worked out independently by enough different teams that naming them feels overdue.

Every pattern names a context, a problem, the forces in tension, a solution, and the context that remains after you apply it. The language reads from root to leaf: *Agentic Loop* is where everything starts, and every later pattern either runs inside it, extends it, or composes several instances of it. The patterns build on each other; there is a reading order but also an applying order.

The thread that runs through the language is this: **a production agent is a negotiation between what the model decides and what your code enforces.** Most of the engineering in a good agent is about knowing which decisions belong to which side. The patterns are, collectively, a vocabulary for that decision.

How to read this

Read straight through if you are new to agent engineering. The order is chosen so each pattern uses only what came before. Skip to a specific pattern if you already know the shape you need; cross-references are explicit.

Patterns are marked with one or two filled stars:

- ★★: a pattern seen stable in multiple independent production systems; the shape is settled.
- ★: a pattern that recurs reliably but whose final shape is still being refined.

Confidence here is about the pattern, not about the idea. Every pattern in this language is one I would reach for today; the stars say how willing I am to claim the shape will not change in a year.

1 Introduction

What agents are, and what goes wrong

An agent is a program that interleaves language-model calls with tool execution, where the model chooses which tool to call and when. That is the whole definition. Everything else (the planning, the memory, the orchestration, the guardrails) is engineering that lives on top of this basic shape.

The first time you build one, the API cooperates. You send a message, you get a response, you parse a tool call, you run the tool, you send the result back, you get a final answer. It works. You ship it. And then the problems start.

The model picks the wrong tool when two tools overlap. The model calls a tool in the wrong order, processing a refund before checking eligibility, because the description said to check first but the model was persuaded otherwise. The model produces a structured output that is technically valid but violates a business rule no schema can express. The model gives a confident answer to a question where it had almost no evidence. A tool times out and the model tells the user the account does not exist. Two sources disagree and the model picks one without saying so. A multi-agent system loses findings in the middle of a long synthesis. Every one of these is a real problem, and every one of them has been solved, independently, more than once, usually by someone who did not know anyone else had solved it.

These problems repeat. The specific tools differ, the domains differ, the frameworks differ. The shape of the problem and the shape of the fix are remarkably stable. Two engineers working on different products, in different companies, in different industries, will arrive at the same three-line class that tracks which tools have succeeded and refuses to let a later tool run until its prerequisites have completed. They will not call it the same thing. But it is the same thing.

That class has a name now. It is *Tool Gate*. This paper names nineteen other things.

What this language is for

A pattern name compresses a paragraph of context into a noun. When someone on a team says *"we need a tool gate here,"* the rest of the team knows what is being proposed, what it costs, what it guarantees, and what alternative was rejected, without re-deriving the design from first principles. That is what the names in this language do.

The language is prescriptive where it can be and cautious where it cannot. The shape of *Agentic Loop* is settled; if you depart from it, you will hit one of three specific bugs I can name from memory. The shape of *Review Pass* is less settled; there are two variants with clear tradeoffs between them and I will tell you which I reach for first, but the space is still moving. Confidence markers (★★ or ★ on each pattern title) say which is which.

Where I am willing to be opinionated, I am. *"Use Self-Evaluation by default; escalate to Critic-Agent when the output ships without human review"* is a stronger claim than *"both variants have merits."* The second is easier to write. The first is more useful.

The thread through the language

One observation keeps surfacing as you apply the patterns. The model has probabilistic authority: given a prompt, a tool set, and a history, it usually, but not always, does the right thing. The surrounding code has deterministic authority: given the same inputs, it does the same thing every time.

A good agent distributes decisions between these two authorities deliberately. *Most of the time* is the right threshold for the rule *"prefer to confirm before processing";* it is not the right threshold for the rule *"refunds above five hundred dollars must be reviewed by a human."* The first belongs in a prompt; the second belongs in code that intercepts the tool call before it runs. The language model is not the problem in the second case. The expectation that the language model would be correct one hundred percent of the time is.

Much of what follows is about finding that line. *Deterministic Override* (Ch. 4) is where code takes over for rules that must hold always. *Tool Gate* (Ch. 5) is where code takes over for orderings that must hold always. *Validation-Retry Loop* (Ch. 6) is where code takes over for structural guarantees the model cannot make on its own. The other patterns live on the probabilistic side: they shape the model's behaviour, observe its uncertainty, route its output by confidence, review its work before delivery. Both sides are necessary. The patterns that step around the model are not better than the patterns that shape it. They are for different problems.

The shape of this paper

A short note on the pattern template (Ch. 2). Then the eight larger patterns, in an order where each uses only what came before: *Agentic Loop* (Ch. 3), *Deterministic Override* (Ch. 4), *Tool Gate* (Ch. 5), *Validation-Retry Loop* (Ch. 6), *Structured Error Classification* (Ch. 7), *Confidence-Based Routing* (Ch. 8), *Review Pass* (Ch. 9), *Coordinator-Subagent Hierarchy* (Ch. 10). Then twelve smaller patterns (Ch. 12), each one a technique that sharpens a larger pattern but is too narrow to carry a chapter. A short closing (Ch. 11). The references chapter gathers everything else.

The paper assumes you have built at least one agent and know the pain. It is not a tutorial, and it is not a literature review. Prior work is named where the naming is load-bearing; the references section gathers everything else.

Sample code is in Python and uses one specific API: Anthropic's, via Amazon Bedrock. The choice is deliberate: a pattern language stays abstract until someone has to implement it, and the translation from concept to working code is where most of the interesting details surface. Picking one API and sticking to it throughout means every code block is a concrete, runnable point of reference that readers can compare against. The patterns are not Claude-specific. Any modern tool-using model exposes enough of the same primitives for every pattern here to apply with small syntactic changes.

Out of scope

Four things this language deliberately leaves alone.

Evaluation. Building an eval *harness* (test-set curation, metric design, CI integration) is a separate discipline and deserves its own treatment. But eval-as-technique runs through several patterns in this language. *Confidence-Based Routing* (Ch. 8) elicits model self-reports. *Review Pass* (Ch. 9) is LLM-as-judge applied to a single output; the rubric discipline is the same discipline you would use for an aggregate eval harness. *Multi-Concern Decomposition* (A.3) pairs with a review rubric. Where these patterns use eval moves, they borrow from the eval literature; the references in those chapters are load-bearing, not decorative.

Adversarial robustness. The patterns assume a cooperative but fallible model. Prompt injection through tool results, jailbreaks, adversarial inputs aimed at the agent: real problems, not this paper's problems. *Deterministic Override* and *Tool Gate* help because they do not trust the model; systematic adversarial security is elsewhere.

Memory across sessions. Every pattern here operates within a single task or conversation. Agents that accumulate context over days, remember users between sessions, or revise long-running plans raise additional questions the language does not address.

Infrastructure economics. Prompt caching, batching policies, rate-limit strategies, model selection per tier: these determine whether any of these patterns is affordable in production. The paper names the patterns; the deployment playbook is separate.

A reading order for the impatient

If you have five minutes, read *Agentic Loop* (Ch. 3). That pattern is the root of everything else.

If you have an hour, read the introduction and Ch. 3 through Ch. 7. Those five patterns (the loop, the two deterministic-guarantee patterns that extend it, the validation-retry loop, and the error taxonomy) cover roughly eighty percent of what a production agent does.

If you have a weekend, read it through. The language is meant to be read linearly once and referenced non-linearly forever.

2 The pattern template

Every pattern in this language follows the same five-move structure, taken from Alexander (1977) and adapted to software:

Context. The kind of system you are building when this pattern becomes relevant. Opens with *"you are building…"* so you know immediately whether you are in the right place. If the context does not describe your situation, the pattern does not apply; skip.

Problem. One or two bolded sentences stating the recurring problem the pattern resolves. The rest of the section shows the problem concretely: what goes wrong when it goes wrong.

Forces. The tensions in play. Nearly every pattern here exists because two things you want are in tension: correctness versus latency, automation versus cost, probabilistic model behaviour versus deterministic guarantees. Naming the forces is what makes the solution feel earned rather than arbitrary. It also tells you when *not* to apply the pattern: if the forces the pattern resolves are not forces in your system, reach for something else.

Solution. A bolded sentence stating what to do, followed by prose that shows how. Sample code lives here. Diagrams live here. This is the longest section of most patterns and the one readers will return to most often.

Resulting Context. What the system looks like after you apply the pattern, and which patterns typically come next. This is the structural move that makes a catalog into a language: every pattern's resulting context is another pattern's context. You find your way through the language by following these threads.

A short *prior art* note at the foot of each pattern cites where the idea has been formalised before, when it has been. Many patterns in this language have no such note; they have been worked out in practice faster than they have been written down.

Confidence markers

Each pattern title carries one or two filled stars:

- ★★: the shape of the pattern is settled. It has appeared in multiple independent production systems with the same essential structure. Changes I would make to its write-up would be refinements, not re-shapes.
- ★: the pattern recurs reliably and is worth naming, but the shape is still being refined. I am confident in the context and the problem; the solution is the best I currently know, and I expect practitioners will produce variants in the next few years that sharpen it.

Alexander used zero, one, and two stars. I have dropped the zero-star tier because a pattern I would not include in a working language does not belong in the language at all. Every pattern here is one I would

reach for; the stars say only how willing I am to claim the shape will not change.

3 Agentic Loop

★★ SHAPE SETTLED

CONTEXT

You are building an agent: a program in which a language model decides what to do next, calls tools, and continues based on what it finds. You have already made the single-call version work: send a prompt, get a response, print it. It handles questions. It does not handle tasks.

The first time you try a task that needs two tool calls where the second depends on the first's result, you discover the shape of the problem. *"Refund the AnyCompany Streaming charge on account ACC-001"* is not one API call. It is *look up the account*, then *find the transaction*, then *check eligibility*, then *process the refund*: four calls, each conditioned on the last. The provider API returns one response per call. Somewhere between those two facts, a loop has to live.

PROBLEM

The model produces intent; your code must produce continuation. Nothing in the API loops on your behalf. If the model's response says "call this tool," the next API call must include that tool's result, or the model cannot reason about what to do next. If the model's response says "I'm done," the loop must stop. Between those two signals, everything is yours to get right.

FORCES

Several things are in tension at once, and they are the reason hand-written loops break in subtle ways.

The model's response is not one thing. It is a list of content blocks (text, tool calls, sometimes more than one of each) and the model expects to see its own previous blocks on the next turn. Appending only the text, which reads naturally to a human, strips out the tool calls and leaves the model amnesic.

Tool results have to be paired to the tool call they answer, by id. The API enforces the pairing: a mismatched or missing `tool_use_id` is rejected as a malformed request. It does not enforce it semantically: a pairing that is structurally valid but semantically wrong produces a model that confidently acts on the wrong information.

When the model emits several tool calls in one response, a sign it expects them to run in parallel, the results have to come back in a single user message, not spread across turns. Splitting them confuses the model's sense of which result answers which call.

The loop has no natural termination. `end_turn` is the only signal that the model is done, and a model that keeps finding new work to do will keep producing `tool_use` responses indefinitely. On an eight-turn task, an unbounded loop runs the first turn's content through the model eight times. On a broken task, it runs it forever.

And the loop has to be shaped for extension. Every later pattern in this language modifies what happens at one specific point in the loop: between the model's tool call and the tool's execution. If you write the loop as a tight function with no seam at that point, every later pattern forces a rewrite.

SOLUTION

Write a loop. Call the model with the message history; if `stop_reason` is `end_turn`, extract the text and return; if `stop_reason` is `tool_use`, execute each requested tool, append the model's full response content as the assistant turn and a single user message containing every tool result, then call again. Cap the iteration count.

Three details carry most of the weight:

Append the entire response content. Not the text block extracted from it. The full list, including every `tool_use` block the model just produced. The model reads back its own previous turn on every iteration and will behave erratically if its tool calls have been deleted from under it.

Match `tool_use_id` to the originating block's `id`. Copy it verbatim from the `tool_use` block; do not regenerate, do not reuse, do not shorten. The pairing is what tells the model which tool call this result answers.

Batch tool results in one user message. When the model emitted three `tool_use` blocks, return one user message with three `tool_result` blocks, not three messages with one each.

Here is the whole thing, in about thirty lines:

PYTHON · RUN_AGENT.PY AGENTIC LOOP

```python
import json
from anthropic import AnthropicBedrock

client = AnthropicBedrock()

def run_agent(user_message: str, max_turns: int = 10) -> str:
    messages = [{"role": "user", "content": user_message}]

    for _ in range(max_turns):
        response = client.messages.create(
            model="global.anthropic.claude-opus-4-6-v1",
            max_tokens=4096,
            system=SYSTEM_PROMPT,
            tools=TOOLS,
            messages=messages,
        )

        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            return next(b.text for b in response.content if b.type == "text")

        if response.stop_reason == "tool_use":
            results = []
            for block in response.content:
                if block.type != "tool_use":
```

```
                continue
            results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": execute_tool(block.name, block.input),
            })
        messages.append({"role": "user", "content": results})
        continue

    raise RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")

raise RuntimeError(f"Exceeded max_turns ({max_turns})")
```

Write this by hand at least once. The invariants above are the kind of detail that only stops being a source of bugs when you have encountered each one in a debugger. Frameworks wrap this loop, and you will use one eventually, but the mental model they expose is this loop, and the stack traces they produce are readable only to people who have seen it.

TRADEOFFS

Token cost grows faster than the loop depth suggests. Every iteration sends the full history again. A ten-turn task sends the first turn's content ten times. Prompt caching (Anthropic, OpenAI, and others all offer it) softens this; without caching, long loops amortise poorly and you will notice on the invoice.

The turn cap is not optional. Pick a number (ten is a reasonable default) and raise a clear error when the loop hits it. Production pipelines that run without a cap eventually find a task that loops forever, and the bill arrives before the alert does.

The loop itself has no place to enforce anything. Rules the model must obey, orderings that must hold, quality checks that must run: none of them live here. They arrive as the next several patterns, each of which changes exactly one step in this loop and leaves the rest alone.

RESULTING CONTEXT

You have a multi-turn agent with honest termination. Whatever you build next lives in its tool-dispatch step.

If one of the tools must not run under certain conditions (a dollar threshold, a compliance rule, a time-of-day restriction) you reach for *Deterministic Override*, which intercepts specific calls before they execute.

If the order of tool calls matters, a refund that must not run before an eligibility check, you reach for *Tool Gate*, which blocks calls whose prerequisites are unsatisfied. Override and Gate together turn the single dispatch step into a three-layer structure: gate, then override, then execute. Both modify this loop; neither replaces it.

If tools can fail in distinct ways, where a mistyped account number is not a timed-out payment processor, you reach for *Structured Error Classification*, the vocabulary those later patterns use to return failures the model can reason about.

If the task is producing structured output rather than dispatching tools, such as extracting findings from a document, you are not in this loop at all. You are in *Validation-Retry Loop*, which shares the shape but operates at a different layer.

And if the task is large enough that one agent accumulates too much context to keep relevance high, you reach for *Coordinator-Subagent Hierarchy*, which nests this loop inside another: a coordinator's loop dispatching subagents whose loops run in parallel.

Everything that follows in this language runs inside some version of what you just wrote. Get this right first.

> ***Prior art.*** The shape of this loop was formalised as ***ReAct*** (Yao et al., 2022). Anthropic's "Building Effective Agents" (2024) distinguishes a "workflow" (scripted pipelines with model calls) from an "agent" (this loop) and places the distinction at the centre of when multi-turn control flow is the right choice.

4 Deterministic Override

★★ SHAPE SETTLED

CONTEXT

You are building an agent that can perform actions with real consequences: process refunds, send emails, modify records, call paid APIs. The agentic loop works. The model usually makes the right call. And "usually" has become a problem.

A rule has appeared in your requirements that must hold every time, not most of the time. *"Refunds above five hundred dollars require human review." "Never send email to addresses on the suppression list." "Block any tool call during a scheduled maintenance window."* The specific rule varies; the shape is constant. There is a line the model must not cross, and the cost of crossing it is not measured in tokens.

PROBLEM

The system prompt tells the model the rule. The tool descriptions tell the model the rule. The model still crosses the line, rarely but repeatedly, and each crossing is expensive. Under unusual inputs, a less capable model variant, a long conversation that pushed the rule out of the attention window, or a customer who phrased the request in a way the prompt did not anticipate, the model calls `process_refund` with an amount of $1,500 and you read about it in the incident report.

FORCES

The rule is probabilistic when you state it to the model, and it needs to be deterministic for your system to be safe. Strengthening the prompt helps for a while, then stops helping. You can reach 99% compliance with a well-written system prompt, and you cannot reach 100% no matter how many words you add.

Moving the rule into the model's environment (fine-tuning, different model, better prompt engineering) does not resolve the force. It moves the failure rate from 1% to 0.1%, which for a rule that must hold always is still broken.

At the same time, not every rule belongs in code. *"Prefer to confirm with the customer before processing"* is a preference that sometimes should not hold, as when the customer has already explicitly authorised, for example. If you encode every preference as an inviolable rule, the agent stops being able to exercise judgment and becomes a decision tree with a language model stapled to the front.

The line between "prompt rule" and "code rule" is: can you name, precisely, in code, the condition under which the rule fires? If you can, and the rule must hold always, it belongs in code. If you cannot state the condition precisely, if the rule involves reading tone, weighing customer intent, judging whether a situation is unusual, it belongs in the prompt.

SOLUTION

Intercept the specific tool call before it executes. When the call meets the predefined condition, replace it with a deterministic result that never reaches the underlying tool. The model sees a tool

result shaped as if the call had proceeded, and continues.

The interception point is the one seam you left in the agentic loop: between the model's `tool_use` block and the call to the tool's implementation. A hook function receives the tool name and input; it returns `None` to let the call proceed, or a result dict to short-circuit.

PYTHON · PRE_TOOL_HOOK.PY — DETERMINISTIC OVERRIDE

```python
ESCALATION_THRESHOLD = 500.00

def apply_pre_tool_hook(tool_name: str, tool_input: dict) -> dict | None:
    if tool_name == "process_refund":
        # Do NOT trust the model's amount. Look it up.
        amount = lookup_transaction_amount(tool_input["transaction_id"])
        if amount > ESCALATION_THRESHOLD:
            return {
                "escalated": True,
                "ticket_id": f"ESC-{tool_input['transaction_id']}",
                "message": (
                    f"This refund of ${amount:.2f} exceeds the automated limit. "
                    f"A specialist will review within 2 business hours."
                ),
            }
    return None  # allow the call to proceed
```

The agentic loop changes at one point:

PYTHON · DISPATCH.PY — OVERRIDE INTEGRATION

```python
# Inside the loop, in the tool_use branch:
hook_result = apply_pre_tool_hook(block.name, block.input)
if hook_result is not None:
    result = json.dumps(hook_result)          # override path
else:
    result = execute_tool(block.name, block.input)  # normal path
```

From the model's perspective, it called a tool and got a response. The response has an `"escalated": true` field, and the system prompt tells it how to narrate that field to the user:

> *When a tool returns `"escalated": true`, explain clearly that the refund exceeded the automated limit and a specialist will handle it. Never expose raw JSON.*

The probabilistic layer (the prompt) is still there. It tells the model how to *respond* to the override. What it no longer does is *enforce* the rule. The hook does that.

The mistake everyone makes. Do not trust the model's parameters in the hook's condition check. A refund tool whose schema marks `amount` as optional may receive `None` from the model, the model's way of saying "full amount." The naive hook checks `tool_input.get("amount", 0) > THRESHOLD`, which evaluates `None > 500` as False, and the $1,500 refund sails through the hook into the tool.

Look the value up from your own source of truth. The ledger knows what the refund amount is; the model's suggestion is a guess. The hook is the deterministic layer; it must derive its condition from deterministic data.

TRADEOFFS

The rule now lives in two places: in the hook (the canonical version) and in the system prompt (so the model does not fight the enforcement). These can drift. The hook wins; the prompt is a shadow that must be maintained in lockstep. Put them in the same file if you can.

The model will still try. The hook prevents the tool from running, not the model from requesting it. Logs will show `process_refund` calls that were overridden. This is a feature, since the signal is useful for operators tracking prompt quality, but it surprises practitioners who expect an enforced rule to also suppress the attempt.

Overuse of this pattern is the failure mode worth worrying about. If every preference is encoded as an override, the agent cannot exercise judgment even where it should. A system where the code intercepts twenty rules has relocated its complexity, not eliminated it, and it will be harder to evolve than the single-agent version it replaced. Use the pattern for rules that genuinely require 100% enforcement. Everything else belongs upstream.

RESULTING CONTEXT

Your agent can now enforce hard rules. The seam it used is available for another pattern to layer into.

The rules you have encoded so far are *conditional*: a specific call, with specific arguments, under specific circumstances. A different class of rule is *ordinal*: "tool A cannot run before tool B has succeeded." Dollar thresholds do not help there; ordinal enforcement needs state across calls, not condition evaluation on a single call. That is *Tool Gate* (Ch. 5). Gate and Override sit at the same seam and compose naturally: Gate checks first, Override checks second, execution runs third.

The override returns a structured result the model must interpret. That result is not an error, but it is shaped the same way structured errors are shaped: a known flag, a message, a category. If you do not yet have a convention for that shape, you will invent one here and then invent another one for tool errors, and they will not compose. *Structured Error Classification* (Ch. 7) is the convention. Adopt it before you have more than one kind of intercept result.

> ***Prior art.*** Policy engines like Open Policy Agent and AWS Cedar are the pre-LLM ancestors: declarative code, consulted before a privileged action runs, producing a deterministic allow or deny. Claude Code's hook system exposes this pattern as a platform primitive; LangGraph's interrupt semantics, the Anthropic Agent SDK's permission system, and AutoGen's approval flows all expose the same shape with different APIs.

5 Tool Gate

★★ SHAPE SETTLED

CONTEXT

You are building an agent whose tools must run in a particular order. Looking up the account must happen before fetching transactions; fetching transactions must happen before checking eligibility; checking eligibility must happen before processing a refund. Every step depends on information the previous step produced. The agentic loop works, and you have written tool descriptions that cross-reference each other, such as *"only call* `process_refund` *after* `check_refund_eligibility` *confirms eligibility,"* and most of the time the model obeys.

PROBLEM

Ordering between tool calls is semantically required, and probabilistic guidance does not guarantee it. Most of the time the model gets the order right. Occasionally, under a less capable model variant, an adversarial customer message, a prompt edit that subtly weakened the instruction, or a model upgrade, the model calls `process_refund` before `check_refund_eligibility`, and the underlying tool either executes a meaningless refund or fails in a confusing way that contaminates the rest of the conversation.

FORCES

Tool descriptions are the natural place to specify ordering, and they work until they don't. The model reads *"only call this after check_refund_eligibility"* and the description raises the compliance rate from unacceptable to almost-acceptable. "Almost-acceptable" is where the pattern starts being needed.

Deterministic Override (Ch. 4) solves a neighbouring problem but not this one. Override evaluates a condition on a single call: *this amount exceeds this threshold*. Ordering is a property across calls: *has the prerequisite already succeeded in this conversation?* Answering that question requires state the hook does not have.

Against the pull toward strict enforcement is the pull toward not re-encoding your entire workflow in code. If your agent has twenty tools and every tool gates every other tool, you have built a state machine with a language model stapled to the front, and you would have been better off writing the workflow as a scripted pipeline. Gates earn their place on a small number of tools whose ordering is genuinely load-bearing. For the rest, the tool description is enough.

A subtler force: success is not the same as completion. A tool that returned, but returned a structured error saying the account was not found, has not produced the information the downstream tool needs. If the gate marks the tool satisfied on any return, a failed account lookup will satisfy the downstream prerequisite and the agent will proceed into nonsense.

SOLUTION

Track which tools have succeeded in the current conversation. Before any tool runs, check its prerequisites against that state. If any are unsatisfied, return a structured error naming them and refuse to execute.

A small class handles both responsibilities:

PYTHON · TOOL_GATE.PY — TOOLGATE CLASS

```python
import json

class ToolGate:
    _prerequisites: dict[str, list[str]] = {
        "get_transactions":         ["get_account_info"],
        "check_refund_eligibility": ["get_account_info", "get_transactions"],
        "process_refund":           ["get_account_info",
"check_refund_eligibility"],
    }

    def __init__(self) -> None:
        self._satisfied: set[str] = set()

    def check(self, tool_name: str) -> str | None:
        missing = [p for p in self._prerequisites.get(tool_name, [])
                   if p not in self._satisfied]
        if missing:
            return (f"Cannot call '{tool_name}': the following tools must "
                    f"succeed first: " + ", ".join(f"'{m}'" for m in missing))
        return None

    def mark_succeeded(self, tool_name: str, result_json: str) -> None:
        try:
            data = json.loads(result_json)
            if not data.get("error", False):
                self._satisfied.add(tool_name)
        except (json.JSONDecodeError, AttributeError):
            self._satisfied.add(tool_name)
```

The gate goes in front of the override hook in the dispatch seam. A tool call now runs through three layers: gate check, override hook, execution. Any of the three can short-circuit.

PYTHON · AGENT_WITH_GATE.PY — THREE-LAYER DISPATCH

```python
def run_agent(user_message: str) -> str:
    messages = [{"role": "user", "content": user_message}]
    gate = ToolGate()   # fresh per conversation

    while True:
        response = client.messages.create(...)
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason == "end_turn":
            return next(b.text for b in response.content if b.type == "text")
```

```
        results = []
        for block in response.content:
            if block.type != "tool_use":
                continue

            # Layer 1: Gate
            gate_error = gate.check(block.name)
            if gate_error:
                result = json.dumps({
                    "error": True,
                    "errorCategory": "prerequisite",
                    "isRetryable": False,
                    "message": gate_error,
                })
            else:
                # Layer 2: Override
                hook_result = apply_pre_tool_hook(block.name, block.input)
                if hook_result is not None:
                    result = json.dumps(hook_result)
                else:
                    # Layer 3: Execute
                    result = execute_tool(block.name, block.input)
                    gate.mark_succeeded(block.name, result)

            results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": result,
            })

        messages.append({"role": "user", "content": results})
```

The gate is instantiated fresh per conversation, inside `run_agent`, never as a module-level singleton. Two users running in parallel must not share gate state.

When the model calls a tool out of order, the gate returns a structured error. The error uses a new category, `prerequisite`, which the system prompt handles:

> *If `errorCategory` is `prerequisite`: a required prior step did not complete. Follow the tool's instruction to call the missing tool first before retrying.*

The model reads the error, recognises the prerequisite, calls the missing tool, and retries. This behaviour is reliable because the error is specific, naming the prerequisite explicitly, and the recovery is obvious.

TRADEOFFS

The prerequisite graph is code that must be maintained alongside the tool set. Add a tool, update the graph. Remove a tool, audit the graph for dangling references. Three tools with a linear chain is trivial; twenty tools with a dense dependency graph is a maintenance burden, and if you find yourself there, question whether the underlying workflow should have been a scripted pipeline rather than an agent.

Content-sensitive ordering ("only process the refund if eligibility returned `eligible: true`") is not what this pattern enforces. The gate enforces *occurrence*, not *content*. If you need the distinction, the eligibility check is a gate concern (the tool must have run) and the eligibility result is an override concern (the content of the result must permit the next call). They compose.

Well-written tool descriptions are still worth the tokens. A gate that rejects every third call because the model keeps mis-ordering is expensive: every rejection is an extra round-trip. Descriptions keep the rejection rate low; the gate makes the remaining violations impossible rather than merely unlikely. Both layers earn their place.

RESULTING CONTEXT

Your agent now enforces both conditional rules (*Deterministic Override*) and ordinal rules (*Tool Gate*) deterministically. Between them, most of what "the model must obey this" means in production is covered.

The gate's rejection result uses a structured shape: `error: true`, `errorCategory: "prerequisite"`, `isRetryable: false`, `message`. If you are returning structured errors from this hook and from your tools, those taxonomies need to match. That is *Structured Error Classification* (Ch. 7), which formalises the shape and is the vocabulary the gate borrows from.

Beyond ordering, tools can fail in ways that have nothing to do with prerequisites: mistyped inputs, frozen accounts, payment processor timeouts. Your prompt currently handles those by guessing from the phrasing of the error string, and it guesses inconsistently. *Structured Error Classification* (Ch. 7) handles them properly; if you have not adopted it yet, now is the time.

> ***Prior art.*** Workflow engines like Temporal and Airflow express prerequisite ordering as a static DAG at the pipeline layer. Tool Gate is the in-conversation analogue: the same shape, applied within a single agent loop rather than across a scheduled pipeline. The Model Context Protocol (Anthropic, 2024) includes tool permissioning that conditionally exposes tools based on session state; Claude Code's per-session permission model is related.

6 Validation-Retry Loop

★★ SHAPE SETTLED

CONTEXT

You are building something different from a tool-using agent: a structured-output pipeline. The task is to take a source document and produce a JSON object that conforms to a schema: extracting findings from research papers, normalising customer support tickets into a data model, converting free-form input into a structured record. There is one API call per document, or close to it. The agentic loop is not what you need; you need a *structure* in the output, reliably, at scale.

Modern APIs offer constrained decoding (`strict: true` on a tool, or the equivalent structured-output mode) and it eliminates a whole class of errors. Types match, enums are valid, required fields are present. You have turned this on. The output is now schema-conformant. And the output is still sometimes wrong.

PROBLEM

The schema can express shape but not meaning, and the errors that matter most are usually the ones about meaning. When the schema requires `source_type` to be one of five enum values but requires `source_type_detail` to be non-null when the value is `"other"` , a cross-field rule JSON Schema cannot express, the model produces `source_type: "other"` with `source_type_detail: null` and the output passes the decoder and fails your system. Retrying with the same prompt produces the same failure. Retrying with *"try again, you got something wrong"* produces a different failure.

FORCES

Schema conformance at decoding is genuinely free on the class of errors it covers. Turning it off because it does not cover everything is the wrong move; leave it on.

Business rules sit above the schema. "This field is required when this other field has this value." "The confidence score must reflect the hedging language in the evidence text." "The evidence must be non-empty." JSON Schema does not reach these; no decoder-level enforcement does. They belong in application code, validated after the decode.

The honest signal for fixing the output is the validator's own error. The message Pydantic produces, *"source_type_detail is required when source_type is 'other',"* is specific enough that a model reading it can identify the offending field and correct it. The generic message *"please try again"* is not. Most of this pattern is about carrying that specificity back to the model without noise.

Unlimited retries are obviously wrong. The source document may genuinely not contain the information you asked for, in which case the model will fail every attempt and the retry budget will be spent on an unfixable input. A retry cap is not optional; two retries past the first attempt is the empirical default.

SOLUTION

Validate the model's output in application code against a richer model than the schema can express. On failure, construct a three-message retry payload: the original user prompt, the assistant's previous

tool call (echoing the bad output back), and a user tool result containing the validator's error with `is_error: true`. Call the model again. Bound the total attempts.

The validator is whatever your language's Pydantic equivalent is. In Python:

PYTHON · FINDING.PY — PYDANTIC MODEL

```python
from pydantic import BaseModel, model_validator
from typing import Literal

class Finding(BaseModel):
    claim: str
    evidence: str
    source_url: str | None = None
    source_name: str
    publication_date: str | None = None
    confidence: Literal["high", "medium", "low", "uncertain"]
    source_type: Literal["academic", "news", "documentation", "blog", "other"]
    source_type_detail: str | None = None

    @model_validator(mode="after")
    def check_source_type_detail(self) -> "Finding":
        if self.source_type == "other" and not self.source_type_detail:
            raise ValueError(
                "source_type_detail is required when source_type is 'other'"
            )
        return self
```

The loop sits around the validator:

PYTHON · EXTRACT_WITH_RETRY.PY — VALIDATION-RETRY LOOP

```python
def extract_with_retry(document: str, max_retries: int = 2) -> SubagentResult |
None:
    raw = extract_once(document)

    for attempt in range(max_retries + 1):
        try:
            return SubagentResult(**raw)
        except ValidationError as e:
            if attempt == max_retries:
                return None  # exhausted; surface upstream

            retry = client.messages.create(
                model="global.anthropic.claude-opus-4-6-v1",
                max_tokens=2048,
                system=SYSTEM_PROMPT,
                tools=[EXTRACTION_TOOL],
```

```
                tool_choice={"type": "tool", "name": "extract_research_findings"},
                messages=[
                    {"role": "user",
                     "content": f"Extract findings from:\n\n{document}"},
                    {"role": "assistant",
                     "content": [{"type": "tool_use",
                                  "id": "prev_extraction",
                                  "name": "extract_research_findings",
                                  "input": raw}]},
                    {"role": "user",
                     "content": [{"type": "tool_result",
                                  "tool_use_id": "prev_extraction",
                                  "content": (f"Your previous extraction failed "
                                              f"validation with these
errors:\n{e}\n\n"
                                              f"Please re-extract, correcting only "
                                              f"the fields mentioned."),
                                  "is_error": True}]},
                ],
            )
            raw = retry.content[0].input

    return None
```

Two details carry the pattern.

Echo the bad output back to the model. The assistant turn in the retry payload contains the failed `tool_use` block: the exact input that was rejected. Without this echo, the model sees an error message with no referent: it does not know what *its own previous output* was, and it guesses at the correction. With the echo, it compares the error against its previous output and corrects the specific field.

Mark the tool result as `is_error: true`. The flag on the tool result is what tells the model that the previous turn was wrong and must be redone, rather than being additional data to incorporate. The same text, returned as a non-error tool result, leaves the model confused about whether it has made a mistake.

The validator's raw message is usually the right thing to send. Pydantic's errors are specific, structured, actionable. Do not paraphrase, do not summarise, do not wrap in a friendly explanation. The specificity is what the model is reading.

TRADEOFFS

Every retry is a full API call. A three-attempt loop triples token cost and triples latency on failing cases. Set the cap low (two is a reasonable default) and instrument the failure rate. A sudden increase in retries is a signal that the source distribution has shifted, the model's capability has regressed, or the schema description is out of date.

Unrecoverable failures consume the full retry budget. When the source document does not contain what the schema is asking for, no amount of retrying helps. A reasonable mitigation: classify the validation

error (schema violations are usually retry-able; domain mismatches are not) and short-circuit for the unretry-able cases. The classification borrows the `isRetryable` flag from *Structured Error Classification* (Ch. 7); you can stop at cap=0 for domain mismatches and cap=2 for shape mismatches.

The loop catches structural and rule-level errors. It does not catch semantic ones. An output that satisfies every rule but whose `claim` is subtly wrong, such as confidence of 0.95 on a hedged statement, or evidence that does not actually support the claim, passes validation and is wrong anyway. *Review Pass* (Ch. 9) and *Confidence-Based Routing* (Ch. 8) address that class of error. The validator does not.

The schema descriptions matter as much as the structure. When you add a business rule, add a description that tells the model about it. A rule enforced silently will keep firing until the model has reason to know it exists.

RESULTING CONTEXT

Your extraction pipeline now produces output you can trust structurally. Everything downstream (analytics, human review interfaces, further transformation) can assume well-formed input. The contract is strong enough to build on.

What the loop did not give you is confidence in the output's *quality*. An extraction can be well-formed and wrong. The `confidence` field on each result carries the model's own uncertainty estimate, which is useful if you do anything with it. *Confidence-Based Routing* (Ch. 8) is the pattern for routing low-confidence output to a reviewer rather than accepting it alongside well-supported claims.

The retry decision, retry or give up, depends on whether the validation error is recoverable. The same question applies to tool errors, and the vocabulary for expressing it is the same. *Structured Error Classification* (Ch. 7) is that vocabulary; if you are not using it for tool errors yet, adopting it now will let validation and tool-use share one error shape.

> ***Prior art.*** The Instructor library (Jason Liu, now maintained by 567-labs) popularised this pattern for the OpenAI tool-use API. OpenAI, Anthropic, and Google now ship constrained-decoding equivalents; these cover the schema-conformance layer. Retry-on-validation remains an application concern across all providers.

7 Structured Error Classification

★★ SHAPE SETTLED

CONTEXT

You are building anything non-trivial on top of tool-using language models. Your tools can fail in more than one way. A customer mistypes an account number; the account lookup returns no result. A payment processor times out; the refund tool cannot complete. An account is frozen for legal hold; the permission check rejects the operation. A prerequisite was skipped; the tool cannot run in the current state. These failures look nothing alike in their causes, and they require nothing alike in response, and your agent currently treats them as one thing.

PROBLEM

When a tool returns an error as a string, the model guesses at what kind of failure it is from the phrasing, and the guesses are wrong often enough to matter. `"Account not found"` produces *"sorry, please try again later,"* which invites another wrong account number. `"Service temporarily unavailable"` produces *"sorry, please try again later,"* which is correct. Same response, one right, one wrong. The model has no systematic way to distinguish them.

FORCES

A single error string collapses meaningfully different situations into one surface. The model's recovery behaviour depends on the nature of the failure (retry for transient, ask the user for validation, explain the constraint for permission, call the missing prerequisite for ordering) and picking the right behaviour from prose is unreliable.

Taxonomies are an API. Once a category is in production use, renaming or removing it is a breaking change across every tool and every system prompt that references it. A taxonomy that starts small and grows is manageable; a taxonomy that starts large is a maintenance burden before it has earned its place. Four categories cover most production agents. Adding a fifth should be a deliberate decision, not a reflex.

Third-party tools often return errors as strings, exceptions, or HTTP status codes. You can wrap them, adapting their errors into your taxonomy at the boundary, but you cannot make the upstream convention conform. The taxonomy is a convention you impose on *your* layer, not a global standard.

The `isRetryable` flag is dangerous when wrong. A non-idempotent tool that returns `isRetryable: true` will cause duplicate side effects on retry. Default it to `false`; set it `true` only when the tool is genuinely idempotent and the failure genuinely leaves the system in a pre-call state. "Transient infrastructure failure" should usually be handled by retries *inside* the tool (with exponential backoff), not surfaced to the agent as retry-able.

SOLUTION

Return errors as structured objects carrying a category, a retry hint, and a human-readable message. Teach the system prompt how to respond to each category. Write one helper that every tool uses.

The shape:

JSON · ERROR PAYLOAD — ERROR SHAPE

```
{
  "error": true,
  "errorCategory": "validation",
  "isRetryable": false,
  "message": "Account 'ACC-INVALID' does not exist. Ask the customer to verify their
account number."
}
```

The helper keeps tools consistent:

PYTHON · TOOL_ERROR.PY — SHARED HELPER

```python
import json

def tool_error(category: str, is_retryable: bool, message: str) -> str:
    return json.dumps({
        "error": True,
        "errorCategory": category,
        "isRetryable": is_retryable,
        "message": message,
    })
```

Four categories cover most agents:

CATEGORY	WHEN TO USE	TYPICAL `ISRETRYABLE`
`transient`	Infrastructure failure (timeout, service unavailable, rate limit) where the tool's own retries are exhausted.	`false` at the agent layer.
`validation`	The input is malformed or does not exist. Mistyped account, missing required field, unparseable date.	`false`.
`permission`	The operation is not allowed: frozen account, insufficient privilege, compliance block.	`false`.
`prerequisite`	A required prior step has not completed. Produced by *Tool Gate* (Ch. 5).	`false`.

Five is reasonable for specific systems: `rate_limited` as a separate category for systems with explicit backoff semantics, `escalated` for systems with human-review workflows. Six is a warning sign. If you

are reaching for a seventh, something else is trying to be expressed; find it.

The system prompt teaches the model what to do with each category:

> *When a tool returns an error object (containing `"error": true`):*
>
> - *`validation`: explain the issue and ask the customer to verify their input.*
> - *`permission`: explain the constraint and suggest an alternative (e.g., contacting support).*
> - *`transient` with `isRetryable: true`: retry the call once.*
> - *`prerequisite`: call the missing tool first, then retry.*
>
> *Never expose raw error JSON. Translate errors into clear, empathetic language.*

And the tools use the helper:

PYTHON · GET_ACCOUNT_INFO.PY — TOOL USING HELPER

```python
def get_account_info(account_id: str) -> str:
    if account_id not in accounts:
        return tool_error(
            "validation", False,
            f"Account '{account_id}' does not exist. Ask the customer to verify "
            f"their account number.",
        )
    account = accounts[account_id]
    if account["status"] == "frozen":
        return tool_error(
            "permission", False,
            f"Account {account_id} is frozen. Refund operations are not permitted.",
        )
    return json.dumps(account)
```

A validation error now produces a specific question back to the customer. A permission error produces an explanation of the constraint. A transient error produces a retry, or, if `isRetryable: false`, an honest acknowledgement that something is not working. The model's behaviour is no longer contingent on reading the mood of an error string.

One related detail worth naming. Not every non-happy outcome is an error. A tool that finds multiple matching records (two accounts registered to the same name, say *"Mateo Jackson"*) has succeeded at searching, but the answer is ambiguous. Use a separate flag: `"ambiguous": true` with a `matches` list, handled by its own prompt rule. Reserving `error: true` for genuine failures keeps the taxonomy clean.

TRADEOFFS

Tool-internal retries versus agent-level retries is a choice worth making explicitly. Most infrastructure failures should be retried *inside the tool*, with backoff, before any error reaches the agent. A 200ms sleep

inside a tool avoids a multi-second round-trip through the model. By the time a `transient` error reaches the agent, the tool's own retry budget is exhausted, at which point the agent's job is to inform the user, not retry further. This pushes toward `isRetryable: false` at the agent layer in most cases.

Taxonomies do not compose cleanly across systems. Two subsystems with their own error categories produce an agent whose system prompt describes both. Prefer one shared taxonomy across the tools your agent uses. When you must integrate with a third-party tool whose errors come in a different shape, wrap at the boundary, adapting the upstream format into your taxonomy, rather than teaching the agent two conventions.

Categorisation is a judgment call. Is an expired access token `permission` or `transient`? Different engineers will answer differently, and the wrong answer routes downstream decisions incorrectly. Document the decisions somewhere other than the tools themselves, or they will drift.

RESULTING CONTEXT

You have a vocabulary that every other pattern in this language uses.

Deterministic Override (Ch. 4) returns a structured result in this shape, often `"escalated": true` rather than `"error": true`, but using the same conventions for fields the prompt reads. *Tool Gate* (Ch. 5) returns errors with `errorCategory: "prerequisite"` and relies on the system prompt's category handling to produce the correction. *Validation-Retry Loop* (Ch. 6) uses the `isRetryable` concept to decide whether a validation error is worth retrying. *Coordinator-Subagent Hierarchy* (Ch. 10) uses the same shape to propagate subagent failures as partial results rather than total failures.

The taxonomy also generalises beyond errors. A *successful* tool result that carries quality metadata (`confidence: 0.62`, or `requires_review: true`) is the same shape applied to success. *Confidence-Based Routing* (Ch. 8) is that application.

Once the convention is in place, the cost of adding another pattern on top of it is very low. Without it, every pattern invents its own result shape and the agent's system prompt becomes a catalogue of special cases.

> ***Prior art.*** HTTP status codes (RFC 9110), with the 4xx/5xx split, are the oldest successful example of a broad error taxonomy on a wire protocol. gRPC's status codes are a richer taxonomy in a modern RPC protocol; many agent error systems read like a subset. The Google SRE book (Beyer et al., 2016) discusses retry budgets and error categorisation in production systems; the `isRetryable` flag descends directly from that tradition.

8 Confidence-Based Routing

★ SHAPE BEING REFINED

CONTEXT

You are running a high-volume structured-output pipeline. The *Validation-Retry Loop* is in place; output is schema-conformant and rule-conformant. Most of the input is straightforward (a named study, a specific statistic, a clear citation) and the model extracts it correctly. A smaller but non-trivial fraction is not: hedged claims, contested findings, ambiguous sources. Human review of every output is too expensive. Human review of none of it is too risky.

PROBLEM

Structural validation cannot tell you which outputs the model is uncertain about, and the outputs the model is uncertain about are the ones that most need a second look. You have no reliable way to separate the ninety percent of straightforward extractions that can ship directly from the ten percent that need a reviewer.

FORCES

Volume and quality are usually in tension. Sending everything to review removes the cost benefit of automation. Sending nothing to review loads the downstream product with output of uneven quality, and in domains like research synthesis, medical summarisation, or legal drafting, uneven quality is the kind of quality that ends careers.

Model-reported confidence is an imperfect signal but a useful one. Kadavath et al. (2022) show that modern language models have meaningful, if miscalibrated, self-knowledge about when they are likely to be wrong. The signal is stronger for well-known topics than for obscure or recent ones, so per-domain calibration is usually better than a single global threshold.

The confidence field collapses if you do not tell the model what it means. Asked for a confidence score without guidance, the model will return 0.95 for everything, because from the model's perspective, it *did* extract the fields correctly, and it interprets confidence as a question about extraction fidelity rather than evidence strength. Explicit scoring guidance in the system prompt turns the field from a formality into a signal.

Confidence scoring does not catch confident hallucinations. A model asserting a false claim with high confidence looks the same at the routing layer as a model asserting a true claim with high confidence. This pattern is triage over *self-reported uncertainty*, not a correctness guarantee. For outputs where high-confidence wrong answers are catastrophic, layer *Review Pass* (Ch. 9) on top.

SOLUTION

Add a `confidence` field to the schema using a small set of named categories, not a 0.0–1.0 float. Teach the model what each category means using linguistic markers it can observe in the source. Route by category: `high` is auto-accept; everything else goes to a review path.

Verbalised confidence on a continuous scale is a known weak point: models are badly calibrated at the granularity of `0.82` vs `0.87` (Tian et al., 2023; Lin et al., 2022) but noticeably better at coarse categorical bins. Four named levels is the sweet spot. Tie each to observable features of the source:

> *Confidence scoring guidelines:*
>
> - `high` *: Claim is directly stated with strong, specific evidence: statistics, named sources, concrete data.*
> - `medium` *: Claim is clearly stated but evidence is general or indirect.*
> - `low` *: Claim uses hedging language ("some researchers suggest", "it is believed") or is explicitly contested.*
> - `uncertain` *: Claim is speculative, anecdotal, or contradicted within the source, or you cannot tell from the text which category fits.*

The routing layer is a small function:

PYTHON · ROUTE.PY — CONFIDENCE ROUTING

```python
from dataclasses import dataclass

AUTO_ACCEPT: set[str] = {"high"}

@dataclass
class RoutingDecision:
    finding: Finding
    requires_review: bool
    reason: str | None = None

def route_finding(finding: Finding) -> RoutingDecision:
    if finding.confidence not in AUTO_ACCEPT:
        return RoutingDecision(
            finding,
            requires_review=True,
            reason=f"confidence '{finding.confidence}' not in auto-accept set",
        )
    return RoutingDecision(finding, requires_review=False)

def route_result(result: SubagentResult) -> tuple[list[RoutingDecision],
list[RoutingDecision]]:
    auto, review = [], []
    for finding in result.findings:
        d = route_finding(finding)
        (review if d.requires_review else auto).append(d)
    return auto, review
```

Which categories count as auto-accept is a policy decision. `{"high"}` is a sensible default; if review capacity is the bottleneck, `{"high", "medium"}` ships more and reviews less. Measure the downstream quality at each setting and pick. Different domains in the same pipeline often warrant different policies: academic findings might auto-accept at `medium` while blog findings require `high`.

A secondary use worth naming. Aggregate category counts turn confidence into an operational metric. Distribution by source type, by extractor version, or by document class produces signals that double as evaluation data over time:

PYTHON · ANALYSE.PY — AGGREGATE CONFIDENCE

```python
from collections import Counter

def analyse_confidence(results: list[SubagentResult]) -> dict:
    by_source: dict[str, Counter] = {}
    for r in results:
        for f in r.findings:
            by_source.setdefault(f.source_type, Counter())[f.confidence] += 1
    return {
        t: {
            "counts": dict(counts),
            "review_rate": round(1 - counts["high"] / max(sum(counts.values()), 1),
3),
            "total": sum(counts.values()),
        }
        for t, counts in by_source.items()
    }
```

If blog-sourced findings land in `low` and `uncertain` while academic-sourced findings mostly land in `high`, the extraction prompt needs more guidance for blog formats. The routing surfaces this for free.

TRADEOFFS

Named categories are not optional: they are the reason the pattern works. A 0.0–1.0 scale looks more sophisticated and is how the pattern is often first drafted, but verbalised continuous confidence is poorly calibrated at sub-tenth precision. Models will return `0.9`, `0.85`, `0.95` for outputs a human would bucket identically. The false precision pushes teams toward hair-trigger thresholds (`> 0.87`) that move with every prompt tweak. Four categories discard information the model never had, and gain routing logic that is stable and explainable.

Confident hallucinations are invisible to this layer. A false claim labelled `high` routes to auto-accept. If that class of failure is expensive, pair this pattern with *Review Pass* (Ch. 9) or retrieval-backed verification; do not rely on confidence routing as your last line of defence.

Calibration drifts. A model upgrade, a prompt change, or a shift in input distribution moves the category distribution without moving your policy. Re-measure after any material change; do not set-and-forget.

Confidence is not a probability. A `high` label is the model's self-report against the rubric, not a frequentist claim. Do not expose it to external consumers as if it were calibrated; it will be read as one and the first counterexample will be a reputational problem.

The review path is a design question this pattern does not answer. Options, in increasing cost: log and drop, annotate and include (let the downstream renderer show uncertainty), human review queue, critic-agent review. For research synthesis, annotating a contested claim as contested is usually more useful than either suppressing or humanly reviewing it. Pick based on the cost of a false negative.

RESULTING CONTEXT

You have cost-and-quality decoupling. Straightforward inputs stay automated. Ambiguous inputs get the review path they need. The threshold is tunable, measurable, and observable.

What you do not yet have is a pattern for *actually reviewing* the low-confidence outputs, or for catching the high-confidence-but-wrong ones. Both belong to *Review Pass* (Ch. 9), which comes next. For low-confidence outputs that need a reviewer, *Review Pass* in its critic-agent variant is the natural fit. For the high-confidence outputs where you still want quality assurance (synthesis reports, decisions that ship without further review) *Review Pass* in either variant is the guard.

The pattern also composes forward into *Coordinator-Subagent Hierarchy* (Ch. 10). A coordinator synthesising findings from multiple subagents should respect confidence: low-confidence findings appear in the final report annotated as contested rather than presented alongside well-supported claims.

Prior art. Content moderation systems (OpenAI Moderation, Google Perspective) routinely route classifier output by threshold into auto-allow, auto-block, and human-review queues. The underlying idea, score every output and branch on the score, is older than language models. What is newer is trusting model self-reports; the empirical basis for that is Kadavath et al. (2022), and Tian et al. (2023) and Lin et al. (2022) show that verbalised confidence is better calibrated in coarse categorical bins than on a 0.0–1.0 scale, the reason this pattern uses named categories rather than floats.

The confidence rubric used here is ★ rather than ★★ because the space is still moving. Per-domain policies, hybrid schemes that blend self-reported confidence with external verifiers, and learned calibrators are all live areas. The pattern's ***shape*** is stable; the specific choices inside it are less so.

9 Review Pass

★ SHAPE BEING REFINED

CONTEXT

You are shipping agent output to users. Tool calls land reliably; the agent answers the question, produces the synthesis, drafts the response. The response is usually fine. Occasionally it is not: not wrong exactly, but incomplete. A customer raised three concerns and the reply addressed two. The refund went through but the reference number the customer will need is missing from the reply. The error happened and the explanation is vague where specific guidance would be better. The tool results contain the information; the draft does not.

PROBLEM

There is a class of errors that no schema and no confidence score can catch. The information is present, the structure is valid, the model is confident, and the output is incomplete or misleading in ways the primary prompt cannot engineer away. Adding more instructions to the primary prompt produces diminishing returns, then fragile prompts.

FORCES

The problem is not the model's reasoning; it is the final rendering. The model knows the three concerns were raised. It addressed two of them. It produced a clean response. If you ask it *"did you address all three concerns?"* it will notice, and it will revise. You just did not ask.

But asking comes at a cost. One extra API call per request doubles the cost of the reviewed turns and adds latency. Apply it indiscriminately and you have made every interaction slower and more expensive to catch a fraction of them.

There are two genuinely different variants with a real tradeoff between them. *Self-Evaluation* asks the same model instance to review its own draft against an explicit rubric. It is cheap (one extra call, same model, same context) and it catches the completeness and formatting errors the primary pass missed. *Critic-Agent* asks a *different* model instance, with a fresh context and a different system prompt, to review the draft. It costs more, but it catches a class of error the Self-Evaluation variant cannot: the kind of error that comes from the primary model having already committed to a narrative. A model asked to review its own synthesis has an implicit prior toward validating it. A model seeing only the output has no such prior.

Shared blind spots are real on both variants. Self-Evaluation cannot catch errors the model does not know to look for, such as factual errors it still believes are correct. Critic-Agent cannot catch errors where the critic and the primary share the same prior (both are wrong about the same obscure fact, or both have the same recency cutoff). For high-stakes factual claims, neither variant substitutes for retrieval-backed verification.

SOLUTION

After the primary pass produces a draft, run a second pass that reviews the draft against an explicit rubric. Pick the variant based on what kind of error you need to catch.

The two variants share a shape but differ in who reviews.

Self-Evaluation variant

The same model receives the draft alongside a checklist. The rubric items are binary and specific to the task:

PYTHON · SELF_EVAL.PY SELF-EVALUATION

```python
EVALUATION_CHECKLIST = """
Review the draft response against this checklist. For each item, answer Yes/No:

1. All concerns addressed: Does the response address every issue the customer
raised?
2. Errors explained: Are all error outcomes explained in plain language?
3. Specific details: Are transaction IDs, amounts, and refund references present?
4. Next steps clear: Does the response tell the customer what to do next?
5. No raw JSON: Is the response free of raw JSON, error codes, or internal
identifiers?

If any item is No, revise the response to address the deficiency.
Return ONLY the final (possibly revised) response text.
"""

def run_agent_with_self_eval(user_message: str) -> str:
    draft = run_agent(user_message)   # primary pass (Agentic Loop)

    eval_response = client.messages.create(
        model="global.anthropic.claude-opus-4-6-v1",
        max_tokens=4096,
        system=EVALUATION_CHECKLIST,
        messages=[{
            "role": "user",
            "content": (f"Original customer message:\n{user_message}\n\n"
                        f"Draft response:\n{draft}"),
        }],
    )
    return eval_response.content[0].text
```

Apply selectively. A lightweight heuristic on the input, checking whether it contains multi-concern indicators like *"and"*, *"also"*, *"additionally"*, routes simple requests straight through and sends complex requests to review. You do not need the review pass on a one-line confirmation.

Critic-Agent variant

A separate agent reviews the draft. The critic's system prompt frames it as an external reader:

PYTHON · CRITIC_AGENT.PY CRITIC-AGENT

```python
CRITIC_AGENT = AgentDefinition(
    description=(
        "Reviews a research synthesis for factual errors, unsupported claims, "
        "and logical inconsistencies. Has no knowledge of how the synthesis "
        "was produced."
    ),
    prompt="""
You are a critical reviewer. You have been given a research synthesis.
You do NOT know how this report was produced — review only what is on the page.

For each claim:
1. Is it supported by the cited source?
2. Is the confidence level appropriate given the evidence?
3. Are there logical inconsistencies between claims?

Return a JSON array of issues:
[
  {
    "claim": "The claim being questioned",
    "issue_type": "unsupported | overstated | inconsistent | missing_citation",
    "description": "Specific reason this claim is problematic",
    "severity": "high | medium | low"
  }
]

If no issues are found, return [].
""",
    tools=[],   # empty; the critic judges what is in the prompt, not what it can discover
)
```

The critic's tools list is empty on purpose. A critic that can re-run searches becomes a second primary pass, not an independent reviewer. The whole point is to judge what is on the page.

The structured issues list makes downstream routing trivial. High-severity issues trigger human review or a primary-pass revision. Low-severity issues are logged. Freeform critique is hard to route.

Choosing between them

Default to Self-Evaluation. Escalate to Critic-Agent when the output ships without further human review, the primary model has a stake in the conclusion, or downstream errors are expensive to detect.

Worth naming explicitly: Review Pass is LLM-as-judge applied to a single output. The rubric discipline below (binary items, task-specific, assessable from the output alone) is the same discipline used in

aggregate eval harnesses (Zheng et al., 2023). A rubric you version for Review Pass should be the same artifact you version for your eval pipeline; keeping two rubrics in sync by hand is how they drift.

Most production agents do not need Critic-Agent. Self-Evaluation catches the common completeness and formatting failures at a fraction of the cost. Critic-Agent is a targeted upgrade for outputs where independence is specifically what you need: usually synthesis tasks where the primary model has committed to a narrative and is poorly placed to spot its own overreach.

TRADEOFFS

Rubric design decides whether the pattern works. Items must be binary (yes/no) so the review is consistent, specific to the task (*"are all refund reference numbers present"* not *"is the response helpful"*), and assessable from the output alone (not *"did we call the right tools,"* which the reviewer cannot see from the response text). Five items is plenty; fifty items is a review pass that itself needs review.

Revision can over-correct. A review told to "revise if deficient" will sometimes rewrite more than it needs to. A prompt clause, *"make the minimum change required; preserve wording that is already correct,"* produces more stable output.

Shared blind spots bound what each variant can catch. Self-Evaluation will agree with the primary model's factual beliefs. Critic-Agent will agree with facts both instances share. For high-stakes factual claims where hallucination is the concern, pair either variant with retrieval, giving the critic a way to verify claims against an external source of truth, rather than hoping a second parametric model will catch errors the first one made.

Rubric drift is a real hazard. Downstream requirements change; the rubric does not. Version the rubric, tie it to the eval harness, and treat it as a first-class artifact. A rubric that stops matching the actual quality bar is worse than no rubric because it produces confident false negatives.

RESULTING CONTEXT

Your agent produces output that meets a stated quality bar, not just a schema bar. The rubric is the spec; when the quality bar changes, you change the rubric rather than re-engineering the primary prompt.

The Self-Evaluation pattern generalises. A rubric applied to a single output is a review pass; the same rubric applied across a test set is an LLM-as-judge evaluation harness. The unit form you built here is the seed of the evaluation system that will tell you whether your next prompt change improved anything.

The Critic-Agent variant is also a preview of *Coordinator-Subagent Hierarchy* (Ch. 10): the critic is a subagent with empty tools, invoked by the primary pass as a review step. If you already have a coordinator pattern, the critic slots in as another subagent with minimal new machinery.

Where this pattern ends. Both variants here sit on a spectrum of reviewer independence. Self-Evaluation shares everything with the primary pass; Critic-Agent shares the model family. The next step along that axis, a jury of heterogeneous models voting on the output, addresses the shared-prior blind spot more directly than either variant here and is an active area of work (Verga et al., 2024). The language does not cover LLM-as-a-jury, but readers should know where Review Pass ends and the jury literature begins.

Prior art. Reflexion (Shinn et al., 2023) formalised self-evaluation with verbal reinforcement learning: the model reviews its own trace, identifies errors, retries with the reflection. Self-Refine (Madaan et al., 2023) is the iterative variant: review, revise, review again, until a quality criterion is met. Constitutional AI (Bai et al., 2022) uses a critic in the training loop, separating the roles of generator and reviewer. Anthropic's multi-agent research system (2024) uses independent critic agents for synthesis review in production. The LLM-as-judge evaluation paradigm (Zheng et al., 2023) is the aggregate form of the Self-Evaluation variant (rubric across a test set rather than a single output) and is the direct ancestor of the rubric discipline used here. Verga et al. (2024) argue for replacing single strong judges with juries of weaker heterogeneous judges, reducing self-preference bias; this is the direction the pattern extends into but does not cover.

This pattern is ★ rather than ★★ because variant selection is still being refined. Hybrid schemes (Self-Evaluation for completeness, Critic-Agent for factuality, Jury for high-stakes factual synthesis, retrieval for anything where ground truth exists) are emerging and will probably supersede the simple two-variant split in a few years.

10 Coordinator-Subagent Hierarchy

★★ SHAPE SETTLED

CONTEXT

You are building an agent to answer questions larger than a single agent can comfortably handle. A research question that spans distinct domains. A customer enquiry that touches several downstream systems. A synthesis task that requires searching, analysing, and reconciling findings from sources that do not share a common format. You have tried making one agent do the whole thing. It works, sort of. The agent's context accumulates past the point where attention stays reliable, the output correlates more with the arrival order of tool results than with their importance, and when one step fails the entire run fails.

PROBLEM

A single agent running a large task loses coherence in three specific ways: context bloat reduces per-token relevance, tool results are processed in arrival order rather than importance order, and partial failures cascade into total failures. Each problem is fixable in isolation; together, they are a signal that the task wants to be decomposed.

FORCES

Decomposition shifts complexity, it does not eliminate it. You trade *one hard agent problem* for *several easier agent problems plus one coordination problem*. If the coordination problem is harder than the original, you have made things worse. The pattern earns its place when sub-tasks are genuinely independent; it does not earn its place when they share context heavily.

Parallel execution is a real win for wall-clock time but not for cost. Four subagents running concurrently cost the same as four subagents running sequentially; you are paying for latency reduction, not for total compute. When one subagent is materially slower than the others, savings cap at the tail latency and you may need to instrument specifically to catch regressions there.

Context isolation is the load-bearing benefit. A subagent scoped to one domain has higher relevance-per-token in its context window than a single agent juggling all domains. Less relevance gets lost; less attention is spent on irrelevant material. The coordinator sees structure but not detail; subagents see detail but not structure. This is the opposite of how a single agent's context behaves at scale.

Failure scoping is the second benefit. A subagent that fails (times out, produces invalid output, surfaces a permission error) contains its failure. The coordinator treats the gap as a gap and proceeds with the rest. A single agent's failure is the whole task's failure; a subagent's failure is one coverage gap in a wider answer.

Coordination is where everything goes wrong. Decomposition is a judgment call the coordinator must make well. Merging is where findings get silently duplicated or silently dropped. Conflict reconciliation is where credible sources disagreeing gets averaged away. Most multi-agent bugs are coordinator bugs.

SOLUTION

Structure the system as a coordinator agent that decomposes the task into sub-tasks with mutually exclusive domains, dispatches subagents in parallel, merges their structured results, preserves rather than reconciles conflicts, and synthesises the final answer. Reach for a framework; do not hand-write the nested loops.

A working coordinator has several moving parts, each earning its place.

Subagent definitions. Each subagent is a narrow agent with its own tools and its own output format. Hard limits on tool calls keep them from over-exploring. Cheaper models are usually fine at this layer; the coordinator does the hard reasoning.

PYTHON · WEB_SEARCH_AGENT.PY — SUBAGENT DEFINITION

```python
from claude_agent_sdk import AgentDefinition

WEB_SEARCH_AGENT = AgentDefinition(
    model="haiku",                    # cheaper model, narrow task
    description=(
        "Searches the web for factual information. Returns structured findings "
        "with source URLs, publication dates, and confidence scores."
    ),
    prompt="""
HARD LIMITS — do NOT exceed:
- 1 WebSearch call
- 2 WebFetch calls
- Then STOP and return JSON immediately

Return ONLY this JSON:
{
  "findings": [{
    "claim": "...", "evidence": "...",
    "source_url": "...", "source_name": "...",
    "publication_date": "YYYY-MM-DD or null",
    "confidence": 0.0-1.0,
    "source_type": "academic|news|documentation|blog|other",
    "source_type_detail": null
  }],
  "gaps": ["Topics I could not find reliable information on"]
}
""",
    tools=["WebSearch", "WebFetch"],
)
```

The coordinator's system prompt. The three clauses doing the real work (parallel emission, domain partitioning, conflict preservation) are explicit. Without them, the coordinator defaults to sequential dispatch and averages-away conflicts:

PYTHON · COORDINATOR_PROMPT.PY — COORDINATOR SYSTEM PROMPT

```
COORDINATOR_SYSTEM = """
You are a research coordinator.

1. Invoke each available subagent exactly once, all in a SINGLE response.
   Emitting all Task calls in one response enables parallel execution.

2. Domain partitioning: assign each subagent a NON-OVERLAPPING domain.
   Example: web-search-agent researches training-time methods (RLHF, data
   quality); doc-analysis-agent researches inference-time methods (RAG,
   prompting). Each subagent prompt explicitly states what it should NOT cover.

3. Conflicting sources: if two sources disagree, preserve both with full
   attribution. Do NOT pick one, average, or discard.

4. Partial failures: if a subagent returns _error or empty findings, note in
   coverage_gaps and proceed with the others.

Return a JSON SynthesisReport with question, findings, conflicts, consensus,
and coverage_gaps.
"""
```

The invocation. One framework call, not a hand-written nested loop:

PYTHON · RUN_PIPELINE.PY — COORDINATOR INVOCATION

```python
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def run_research_pipeline(question: str) -> dict:
    final_result = None
    async for message in query(
        prompt=f"Research this question: {question}",
        options=ClaudeAgentOptions(
            system_prompt=COORDINATOR_SYSTEM,
            permission_mode="bypassPermissions",
            allowed_tools=["Task", "Read", "Glob"],
            agents={
                "web-search-agent": WEB_SEARCH_AGENT,
                "doc-analysis-agent": DOC_ANALYSIS_AGENT,
            },
            max_turns=10,
        ),
    ):
        if isinstance(message, ResultMessage):
            final_result = json.loads(message.result)
    return final_result
```

Coordinator-side post-processing. Two responsibilities the coordinator performs in code, not in prompt:

PYTHON · POST_PROCESS.PY — CONFLICT + REORDER

```python
def detect_conflicts(findings: list[Finding]) -> list[ConflictRecord]:
    topic_groups: dict[str, list[Finding]] = {}
    for finding in findings:
        topic_key = " ".join(finding.claim.lower().split()[:5])
        topic_groups.setdefault(topic_key, []).append(finding)

    conflicts = []
    for topic, group in topic_groups.items():
        if len(group) < 2:
            continue
        sources = {f.source_url for f in group if f.source_url is not None}
        if len(sources) < 2:
            continue        # same source — not a real conflict
        if all(f.confidence > 0.5 for f in group):
            conflicts.append(ConflictRecord(
                topic=topic,
                conflicting_claims=[f.model_dump() for f in group],
                resolution="unresolved",
            ))
    return conflicts

def reorder_for_synthesis(findings: list[Finding]) -> list[Finding]:
    """Place highest-importance findings at the beginning and end."""
    ranked = sorted(findings, key=lambda f: f.confidence, reverse=True)
    half = len(ranked) // 2
    front, back = ranked[:half], ranked[half:]
    return front + list(reversed(back))
```

Conflict detection runs before synthesis, not after. Reordering addresses the position-sensitivity of large-context synthesis, a real effect documented by Liu et al. (2023) that the pattern would otherwise leave on the table.

Reach for a framework. Writing the nested loops by hand is possible. Writing them correctly (concurrent dispatch, message plumbing across tiers, lifecycle management for subagent processes, error handling that distinguishes subagent crashes from subagent errors) is the kind of work where frameworks earn their place. The Anthropic Agent SDK, LangGraph's supervisor patterns, and AutoGen's group-chat variants all ship this machinery. Use one. The mental model you built in *Agentic Loop* (Ch. 3) is still the accurate picture of what the framework is doing; hand-rolling it for multi-agent work adds engineering without adding understanding.

TRADEOFFS

Most failures in multi-agent systems are coordinator failures. Decomposition, reconciliation, and conflict handling are the hard parts, and none of them are solved by the framework. When a multi-agent pipeline

behaves badly, inspect the coordinator's prompt first, not the subagents.

Duplicate findings are the clearest sign of a coordinator bug. If two subagents return overlapping work, the coordinator either failed to assign non-overlapping domains or assigned them too loosely. The fix is domain partitioning at dispatch time: explicit boundaries in each subagent's prompt, not deduplication at synthesis. See *Domain Partitioning* (A.7).

Parallel execution reduces latency but not cost. Four subagents running concurrently run four API calls. Scale your cost model accordingly and instrument tail latency specifically; a single slow subagent silently eats the parallel gains.

The cost multiplier is a real consideration in its own right. A pipeline that runs four subagents concurrently costs roughly four times what a single agent answering the same question would cost. The right question is not "am I saving money" but "am I getting answers I could not otherwise get, at a cost I can justify."

Lost-in-the-middle effects are a real phenomenon, not a detail. Large synthesis inputs degrade in quality when high-importance findings land in the middle of the input. *Lost-in-the-Middle Mitigation* (A.11) is the post-processing step; the reordering function above is its minimal form.

Partial failure is an engineering choice, not a technology limitation. A pipeline that fails totally when any subagent fails is fragile in a way that has nothing to do with the model. Design for partial results from the start: coverage gaps are first-class output, not exceptions.

RESULTING CONTEXT

You have a system that handles tasks too large for a single agent, parallel-dispatches its sub-tasks, and produces honest partial output when something fails. The language of the smaller patterns (*Domain Partitioning*, *Parallel Execution*, *Error Propagation with Partial Results*, *Conflict Detection*, *Lost-in-the-Middle Mitigation*) fills in the specific implementation moves inside this pattern. They do not stand on their own; they live inside this one.

Review is a natural extension. The *Review Pass* (Ch. 9) pattern in its Critic-Agent variant slots in as another subagent: empty tools, fresh context, reviewing the coordinator's synthesis before delivery. If your output ships without human review, the critic belongs here.

The coordinator-subagent shape is one topology among several. *Peer-to-peer swarms* (where agents hand off to each other without a central coordinator) trade failure-scoping for more fluid task flow. *Graph-topology coordination* (subagents dispatching their own subagents, or conditional routing between peers) generalises the star shape this pattern uses. Both are active areas; neither is settled enough to sit alongside the patterns here.

You are also now at the edge of this language. Long-running agents that accumulate memory across sessions, persistent goals pursued over days, adversarial robustness against hostile inputs: these are problems beyond what the current patterns address. The closing names what is out of scope.

Prior art. Anthropic's "How we built our multi-agent research system" (2024) is the canonical production write-up; the shape described there is close enough to this pattern that the differences are editorial rather than substantive. AutoGen (Wu et al., 2023) frames multi-agent coordination as a conversational protocol: the GroupChat-with-Manager variant is this pattern. LangGraph's supervisor patterns expose the coordinator-subagent shape as a graph topology. Academic multi-agent work, such as Park et al. (2023) on generative agents and Qian et al. (2023) on ChatDev, demonstrates generalisation to simulation and software-engineering tasks.

11 Closing

The eight larger patterns and twelve smaller ones in this language are a starting vocabulary, not a finished one. What they give you is enough words to have the design conversations that currently get re-derived in every new project. Someone says *"we need a tool gate here"* and the team knows what is being proposed, what it costs, and what is being rejected. The patterns compress decisions. That is the whole purpose.

The thesis the language hangs on, that agent engineering is a negotiation between what the model decides and what your code enforces, is the move worth remembering when you apply the patterns in a situation they do not exactly fit. The question *"does this decision belong to the model or to code?"* is useful far beyond the specific patterns in this paper. Most of the engineering in a good agent system is about knowing where the answer is *code*, and the patterns here are ways of writing that answer down.

What this language does not cover

Four kinds of problem this paper deliberately leaves alone.

Evaluation. Building an eval *harness* is a separate discipline, but eval-as-technique is embedded in several patterns here: *Confidence-Based Routing* (Ch. 8), *Review Pass* (Ch. 9), *Multi-Concern Decomposition* (A.3). The references in those chapters are load-bearing. The harness itself (test-set curation, metric design, CI integration) needs its own paper.

Adversarial robustness. The language assumes a cooperative but fallible model. Prompt injection through tool results, jailbreaks, agents under hostile input: real problems, different shape. *Deterministic Override* and *Tool Gate* help because they stop trusting the model; systematic adversarial security is elsewhere.

Long-horizon agents. Everything here operates within a single task or conversation. Agents that accumulate memory over sessions, revise plans over days, pursue goals that span weeks: these raise problems the language does not address. Memory compaction, cross-session reference resolution, plan revision all need their own patterns.

Deployment economics. Prompt caching, batching, rate-limit strategy, per-tier model selection: the patterns name what to build; the playbook for running it cheaply is separate and changes faster than the patterns do.

Using this language

Three practical notes for readers reaching for it.

Start with *Agentic Loop* by hand. Even if production uses a framework, the understanding compounds. The three invariants (full content appending, matched `tool_use_id`, batched results) are the kind of

thing you absorb only after one debugging session with them.

Reach for deterministic guarantees sparingly. Each of *Deterministic Override*, *Tool Gate*, and *Validation-Retry Loop* introduces code that must be maintained alongside the probabilistic layer. Use them where the cost of a violation justifies the burden. A system that uses them everywhere has relocated complexity rather than eliminating it.

Extend the language. The twenty patterns here are a starting point, not a closed set. New patterns will emerge as agent engineering matures, and some of the ★ patterns here will be refined beyond recognition. A living language is one that grows; if you find yourself writing a pattern that fits the five-move shape and has occurred to you three separate times, you have probably found a new one. Name it.

The next few years of agent engineering will produce better patterns than these. That is the point. A language is useful while it is current, and writing one down is how the next one gets built.

12 Smaller patterns

The twenty patterns in this language vary in size. Twelve of them are narrower than the eight in the main sequence: each one a move or technique that sharpens a larger pattern, not a standalone shape that can carry a chapter of its own. They appear here in a compressed form: context, problem, solution, and the pattern they extend.

Most of them are ★★: the shapes are settled, even if the containing patterns they modify are still moving. Where the shape is less stable, I say so.

A.1 Tool Description Disambiguation

★★ SHAPE SETTLED

You are using more than two tools, and some have overlapping surface area. `get_account_info` and `get_account_summary`. `search_documents` and `query_knowledge_base`. The model picks the wrong one often enough to notice.

The problem is that a tool description in isolation tells the model what the tool does but not what it is *not for*. The model reads a plausible description and picks a plausible tool; if two tools are both plausible, it picks by coin flip.

Cross-reference and use negative instructions. *"Use this tool for X. Do not use it for Y; that is handled by `other_tool`."* Two or three such cross-references per tool catch most of the disambiguation cases. More than that produces diminishing returns: descriptions compete with every other instruction in the system prompt.

The technique is probabilistic; it raises compliance without guaranteeing it. When a rule must hold always, escalate to *Tool Gate* (Ch. 5). For the common case, the model picking a slightly wrong tool on edge inputs, descriptions are the cheapest fix with the highest marginal return.

> *Extends: Agentic Loop (Ch. 3).*

A.2 Session Facts

★★ SHAPE SETTLED

You are running a multi-turn conversation. Facts confirmed in early turns (the customer's account ID, the choice of product variant, the verified identity) must remain unambiguously available on every subsequent turn. Context compression, summarisation, or long histories erode the model's ability to retrieve them reliably from the conversation.

The facts the model needs to treat as ground truth should not be buried inside message history. History is summarised, compressed, and occasionally pruned. None of those operations preserve specific facts well.

Inject confirmed facts into the system prompt, not the message history. The system prompt is rebuilt on every turn and cannot be compressed out from under you. A short, named list of session facts, such as *"Confirmed: account_id=ACC-001; verified_email=yes; product_tier=premium,"* remains visible regardless of how the rest of the conversation is managed.

The discipline is keeping the facts list narrow. A session fact that carries yesterday's context into today's conversation is a bug. Facts scoped to the current task, written by code (not by the model), cleared when the task ends.

Extends: Agentic Loop (Ch. 3).

A.3 Multi-Concern Decomposition

★ SHAPE BEING REFINED

Users send messages that bundle several concerns. *"I need a refund on the duplicate charge, and I want to know why my tier hasn't been upgraded, and can you also look at the failed AnyCompany Streaming payment?"* The agent addresses two of the three, or merges them in ways that cross-contaminate parameters.

Models do not naturally enumerate concerns before acting. They pick one, proceed, and occasionally forget the others. The tool results cover all three issues; the final response does not.

Instruct the model to decompose before acting. *"Before calling any tool, identify each distinct concern and label its parameters. Only use parameters bound to a concern when calling tools for that concern."* Combine with a *Review Pass* (Ch. 9) rubric item (*"does the response address every issue the customer raised?"*) to catch the cases where the decomposition held but the final rendering dropped one.

The technique is probabilistic. For concerns that must be addressed without exception, whether regulatory or safety-critical, pair with review or structured output. For normal customer interactions, decomposition plus review is almost always enough.

This pattern is ★ rather than ★★ because the exact phrasing of the decomposition instruction is still moving. The shape is stable; the wording will be sharpened as models improve at self-enumeration.

Extends: Agentic Loop (Ch. 3); pairs with Review Pass (Ch. 9).

A.4 Nullable Fields

★★ SHAPE SETTLED

You are extracting structured output from source documents. Some fields are legitimately optional in the source: publication date may be missing, author may be unattributed, URL may not appear.

If the schema marks a field required, the model will fill it even when the source does not contain the information, because the schema has told it the field must be present. The fill is plausible and wrong.

Declare optional fields as nullable. Give the model an honest way to say *"this source did not say."* Pair with a system-prompt instruction (*"Return null for any field where the information is not present in the source. Do not infer or fabricate values"*) so the model understands null as the correct answer rather than a failure.

Every nullable field is a check in downstream code, so do not nullable-ify promiscuously. Make fields nullable when absence is semantically meaningful (a source without a URL is still a valid finding). Keep them required when absence would be an extraction error (a finding without a claim is broken).

Extends: Validation-Retry Loop (Ch. 6).

A.5 Other + Detail Enum

★★ SHAPE SETTLED

You have an enum field (`source_type` , `error_category` , `product_kind`) that covers the common cases cleanly but occasionally meets a value outside the canonical set. You need the cleanliness of enums for normal processing and a legitimate fallback for the edge cases.

Without a fallback, the model either picks the closest-wrong enum value or reports an error. Both are worse than admitting the source does not fit the categories.

Pair the enum with a free-text `*_detail` field, required when the value is `other` . Five fixed values plus one escape hatch. The cross-field rule (`source_type_detail` must be non-null when `source_type == "other"`) is enforced by *Validation-Retry Loop* (Ch. 6), because JSON Schema cannot express it.

Watch the rate of `other` values. A steady stream of them is a signal that your enum is stale and needs a new category. The `*_detail` field is a diagnostic surface, not a permanent bucket.

> *Extends: Validation-Retry Loop (Ch. 6).*

A.6 Few-Shot Format Examples

★★ SHAPE SETTLED

You are extracting from heterogeneous sources: academic papers, news articles, blog posts, documentation. The same field is encoded differently across source types: dates as "March 2024" versus "2024-03-15", authors as bylines versus affiliations, claims as abstracts versus pull quotes. Extraction consistency suffers.

Descriptions alone tell the model what you want; they do not tell it what each source format looks like.

Include one example per format. A short extraction example for each source type you expect, placed in the prompt or few-shot position. The examples demonstrate how the schema fields map to each format's conventions; the model generalises.

Token cost is real: every example adds context on every call. Include one per format, not ten. Prioritise formats that produce extraction errors; retire examples that no longer earn their cost. Well-written schema descriptions plus strict tool use substitutes for many cases; examples are the fix for the cases descriptions cannot carry.

Extends: Validation-Retry Loop (Ch. 6).

A.7 Domain Partitioning

★★ SHAPE SETTLED

You are running a coordinator with multiple subagents. The subagents return overlapping findings, presenting duplicate work as independent corroboration.

Deduplication at synthesis is expensive and error-prone; preventing overlap at dispatch is trivial and exact.

State non-overlapping domains explicitly in each subagent's prompt. *"Your domain is training-time methods: RLHF, data quality. Do NOT cover inference-time methods; that is handled by another subagent."* Each subagent knows what it covers and what it does not. Overlap becomes impossible at the source rather than something you clean up downstream.

The coordinator must know the domain structure well enough to partition it. For exploratory questions where the natural decomposition is not obvious, dynamic partitioning (the coordinator decides per-dispatch) is more robust than static, at the cost of less predictable output.

Extends: Coordinator-Subagent Hierarchy (Ch. 10).

A.8 Parallel Execution

★★ SHAPE SETTLED

The coordinator is dispatching subagents sequentially (call, wait, call, wait) producing wall-clock latency equal to the sum of subagent latencies.

Frameworks support concurrent dispatch; coordinators default to sequential emission without explicit instruction.

Instruct the coordinator to emit all subagent calls in a single response. *"Invoke each available subagent exactly once, all in a SINGLE response. Emitting all Task calls in one response enables parallel execution."* The framework runs them concurrently; wall-clock time drops to the max of subagent latencies plus a small coordination overhead.

Parallel execution reduces latency, not cost: the number of API calls stays constant. Instrument tail latency; a single slow subagent silently caps the savings. The workshop's two-subagent pipeline sees roughly 20% latency reduction; wider pipelines see more, up to where the slowest subagent dominates.

Extends: Coordinator-Subagent Hierarchy (Ch. 10).

A.9 Error Propagation with Partial Results

★★ SHAPE SETTLED

A subagent in your multi-agent pipeline fails: times out, returns invalid output, surfaces a structured error. The whole run fails.

Total failure on partial cause is an engineering choice, and almost always the wrong one. Multi-agent pipelines should degrade, not fall over.

Annotate the failure as a coverage gap with a failure-type tag; continue with the remaining subagents' findings. A timeout is not the same as an empty result is not the same as an invalid output; distinguish them in the gap record. The final synthesis is honest about what was and was not covered: *"I could not retrieve X; here is what I found for Y and Z."*

The honest framing is the load-bearing discipline. The dishonest alternative, presenting Y and Z as if the absence of X were not a gap, is the production-quality mistake this pattern prevents. Preserving the appearance of completeness is not worth the degradation in trust once a user notices.

Extends: Coordinator-Subagent Hierarchy (Ch. 10); uses Structured Error Classification (Ch. 7) for the taxonomy.

A.10 Conflict Detection

★★ SHAPE SETTLED

Your synthesis pipeline is consuming findings from multiple subagents. Two credible sources report different statistics for the same phenomenon. The naive synthesis picks one, averages them, or omits the disagreement entirely.

Credible sources disagree. An agent that hides disagreement produces a final report more confident than its evidence supports.

Group findings by topic; flag pairs where multiple sources make incompatible claims. A cheap topic key (the first few words of the claim, normalised) plus a source-distinctness check plus a confidence floor (both findings above 0.5, say) identifies real conflicts. Preserve them in the output under an explicit `conflicts` array rather than merging them into the consensus findings.

Conflicts need resolution or deferral. Listing disagreements without guidance is not useful. Pair with *Review Pass* (Ch. 9) in the Critic-Agent variant, where the critic can judge which position has stronger evidence, or surface the conflict to a human reviewer. What you do not do is pick one and present it as consensus.

Extends: Coordinator-Subagent Hierarchy (Ch. 10).

A.11 Lost-in-the-Middle Mitigation

★★ SHAPE SETTLED

Your coordinator is synthesising many findings: tens, maybe hundreds, aggregate input over 20,000 tokens. The quality of the synthesis correlates with the *position* of high-importance findings rather than their content.

Attention in large-context models is not uniform across input. Liu et al. (2023) document that content near the beginning and end of long inputs is attended to reliably; content in the middle is under-weighted.

Before synthesis, reorder findings so the most important occupy the edges. Highest-confidence first, second-highest last, mid-importance in the middle. Confidence is convenient; a domain-specific importance score is better if you have one.

The effect's magnitude varies by model and by input shape. Measure before investing; the pattern is most valuable for very large syntheses (80,000+ tokens), less important for shorter ones. For inputs under 20,000 tokens, the reordering rarely moves the needle.

Extends: Coordinator-Subagent Hierarchy (Ch. 10).

A.12 Strict Tool Use

★★ SHAPE SETTLED

You are producing structured output. The provider API supports a constrained-decoding mode (`strict: true` on tools, or the equivalent structured-output feature) and you have not yet turned it on.

Schema conformance at the decoding layer is a free reliability improvement that no amount of application-level retry can match. Type mismatches, invalid enum values, missing required fields: all gone, at zero incremental cost per call.

Enable strict tool use wherever your provider supports it for your target model. OpenAI, Anthropic, and Google all offer equivalents. Pair with *Validation-Retry Loop* (Ch. 6) for the business-rule layer above the schema; strict decoding handles what can be expressed as schema, and the retry loop handles what cannot.

Two-layer validation is the correct default for any structured-output pipeline. Strict mode at decoding plus Pydantic (or equivalent) above it catches every error class worth catching, at minimum cost.

Extends: Validation-Retry Loop (Ch. 6).

the References References

Pattern-language foundations

Alexander, C., Ishikawa, S., Silverstein, M., Jacobson, M., Fiksdahl-King, I., & Angel, S. (1977). *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press. The template, the confidence markers, and the idea that a catalog becomes a language when its entries chain into one another all come from this book.

Agents, tool use, and the agentic loop

Anthropic. (2024). *Building effective agents*. <https://www.anthropic.com/research/building-effective-agents>. The workflow-versus-agent distinction referenced in *Agentic Loop* (Ch. 3).

Anthropic. (2024). *How we built our multi-agent research system*. <https://www.anthropic.com/engineering/multi-agent-research-system>. Production reference for *Coordinator-Subagent Hierarchy* (Ch. 10).

Anthropic. (2024). *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>. Tool-permission model referenced in *Tool Gate* (Ch. 5).

Anthropic. (2025). *Claude Code: hooks reference*. <https://code.claude.com/docs/en/hooks>. Production example of pre-tool-use interception.

Anthropic. *Tool use with Claude* (API documentation). <https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview>. `stop_reason` semantics, `tool_use` / `tool_result` block protocol.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv:2302.04761*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing reasoning and acting in language models. *arXiv:2210.03629*. The academic ancestor of *Agentic Loop* (Ch. 3).

Multi-agent systems

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *UIST 2023*. <https://arxiv.org/abs/2304.03442>.

Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., & Sun, M. (2023). ChatDev: Communicative agents for software development. *arXiv:2307.07924*.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent

conversation. *arXiv:2308.08155*.

Review, calibration, and self-correction

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, D., Tran-Johnson, E., Perez, E., … Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*. The critic-in-the-loop structure that the Critic-Agent variant of *Review Pass* (Ch. 9) borrows from.

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. *ICML 2017*. <https://arxiv.org/abs/1706.04599>. Background for *Confidence-Based Routing* (Ch. 8).

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., … Kaplan, J. (2022). Language models (mostly) know what they know. *arXiv:2207.05221*. Empirical basis for treating model self-reported confidence as a useful signal: the core justification for *Confidence-Based Routing* (Ch. 8).

Lin, S., Hilton, J., & Evans, O. (2022). Teaching models to express their uncertainty in words. *arXiv:2205.14334*. Evidence that verbalised confidence is better calibrated when elicited in discrete words (e.g. "unlikely", "likely") than as a continuous probability: justification for the categorical rubric in *Confidence-Based Routing* (Ch. 8).

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Prasad Majumder, B., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. *arXiv:2303.17651*. The iterative variant of *Review Pass* (Ch. 9).

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *arXiv:2303.11366*. Academic formulation of the Self-Evaluation variant of *Review Pass* (Ch. 9).

Tian, K., Mitchell, E., Zhou, A., Sharma, A., Rafailov, R., Yao, H., Finn, C., & Manning, C. D. (2023). Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. *arXiv:2305.14975*. Shows that verbalised confidence from RLHF-tuned models is poorly calibrated at fine numeric granularity but recovers under coarse categorical elicitation: the empirical grounding for using named categories in *Confidence-Based Routing* (Ch. 8).

Verga, P., Hofstatter, S., Althammer, S., Su, Y., Piktus, A., Arkhangorodsky, A., Xu, M., White, N., & Lewis, P. (2024). Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv:2404.18796*. Argues that juries of weaker heterogeneous judges outperform single strong judges and reduce self-preference bias: the direction *Review Pass* (Ch. 9) extends into but does not cover.

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv:2306.05685*. The aggregate (evaluation-over-test-set) form of *Review Pass*.

Context, retrieval, and position effects

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*. <https://arxiv.org/abs/2005.11401>.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2023). Lost in the middle: How language models use long contexts. *arXiv:2307.03172*. The grounding reference for *Lost-in-the-Middle Mitigation* (A.11).

Structured output and validation

Anthropic. *Structured outputs on Amazon Bedrock* (tool use with `strict: true`). <https://docs.aws.amazon.com/bedrock/latest/userguide/structured-output.html>.

Liu, J. *Instructor* (maintained by 567-labs). <https://github.com/567-labs/instructor>. The Python library that popularised *Validation-Retry Loop* (Ch. 6) for the OpenAI tool-use API.

OpenAI. (2024). *Introducing Structured Outputs in the API*. <https://openai.com/index/introducing-structured-outputs-in-the-api/>. Provider-level schema enforcement at decoding time; referenced in *Validation-Retry Loop* (Ch. 6) and *Strict Tool Use* (A.12).

Pydantic. *Pydantic documentation*. <https://pydantic.dev/docs/>. The validation library used in the *Validation-Retry Loop* (Ch. 6) sample code.

Structured error taxonomies

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (Eds.). (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media. Retry-budget discipline and taxonomised failures in production systems: the `isRetryable` flag in *Structured Error Classification* (Ch. 7) descends directly from this tradition.

Fielding, R., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). IETF. <https://www.rfc-editor.org/rfc/rfc9110>. The 4xx/5xx status-code families: the oldest broad error taxonomy in a wire protocol.

gRPC Authors. *gRPC status codes*. <https://grpc.io/docs/guides/status-codes/>. A richer error taxonomy in a modern RPC protocol.

Policy engines

Amazon Web Services. *Cedar policy language*. <https://www.cedarpolicy.com/>.

Cloud Native Computing Foundation. *Open Policy Agent*. <https://www.openpolicyagent.org/>.

Both are pre-LLM ancestors of *Deterministic Override* (Ch. 4): declarative code consulted before a privileged action runs.

Agent frameworks

Anthropic. *Claude Agent SDK (* `claude-agent-sdk` *)*. <https://github.com/anthropics/claude-agent-sdk-python>. Used in the *Coordinator-Subagent Hierarchy* (Ch. 10) sample code.

LangChain Inc. *LangGraph documentation*. <https://docs.langchain.com/oss/python/langgraph/overview>. Supervisor patterns referenced in *Coordinator-Subagent Hierarchy* (Ch. 10).

Source workshop

The sample code and several of the scenarios in this language are drawn from the workshop *Agentic Design Patterns with Claude*, built for Amazon Web Services. The workshop's three exercises (a multi-tool agent with escalation logic, a research subagent output layer, and a multi-agent research pipeline) contain runnable implementations of every larger and smaller pattern in this paper.